DSP-Efficient Hardware Acceleration of Convolutional Neural Network Inference on FPGAs

Dong Wang, *Member, IEEE*, Ke Xu, Jingning Guo, and Soheil Ghiasi, *Senior Member, IEEE*

***Abstract*—Field-programmable gate array (FPGA)-based accelerators for convolutional neural network (CNN) inference have received significant attention in recent years. The reported designs tend to adopt a similar underlying approach based on multiplier-accumulator (MAC) arrays, which yields strong demand for the available on-chip DSP blocks, while leaving FPGA logic and memory resources underutilized. The practical outcome is that the computational roof of the accelerator is bound by the number of DSP blocks offered by the target FPGA. In addition, integrating the CNN accelerator with other functional units that may also need DSP blocks would degrade the inference performance. Leveraging the robustness of inference accuracy to limited arithmetic precision, we propose a transformation to the convolution computation, which leads to transformation of the accelerator design space and relaxes the pressure on the required DSP resources. Through analytical and empirical evaluations, we demonstrate that our approach enables us to strike a favorable balance between utilization of the FPGA on-chip memory, logic, and DSP resources, due to which, our accelerator considerably outperforms state of the art. We report the effectiveness of our approach on a variety of FPGA devices, including Cyclone-V, Stratix-V, and Arria-10, which are used in large number of applications, ranging from embedded settings to high performance computing. Our proposed technique yields 1.5× throughput improvement and 4× DSP resource reduction compared to the best frequency domain convolution-based accelerator, and 2.5× boost in raw arithmetic performance and 8.4× saving in DSPs compared to a state-of-the-art sparse convolution-based accelerator.**

***Index Terms*—Accelerator architectures, application specific integrated circuits, artificial neural networks, neural network hardware, reconfigurable architectures.**

I. Introduction

CONVOLUTIONAL neural network (CNN) has become the dominate approach in many artificial intelligence (AI) applications, such as computer vision [5], speech recognition, and robotics [6]. Many studies [2] have been carried out to design various CNN hardware accelerators for real-time processing. Being able to provide massive computational resources with flexible data precision, lower power dissipation and shorter deployment cycle, field-programmable gate array (FPGA)-based CNN inference accelerator has received great attention in various application fields from large-scale data-centers [4] to energy-constrained Internet of Things (IoT) devices [5].

One of the key challenges in designing high performance FPGA-based CNN accelerator is to take full advantage of the on-chip computing resource (i.e., logic and DSP) to speedup multiply accumulate (MAC) operations involved in the convolution (CONV) and fully connected (FC) layers, which account for over 99% [3] of the total operations for most CNNs. According to the type of convolution algorithm implemented, one can divide existing FPGA-based inference accelerators into three broad categories. The first kind of designs, such as [9]–[15], directly exploit parallelism of the convolution computation in *spatial domain* (referred to as SDConv) by performing massive number of MAC operations on a large array of DSP blocks in every cycle. As shown in Fig. 1, the design space of SDConv-based accelerator has a computational roof of $2 \times N_{\text{mac}} \times \text{Freq}$, where N_{mac} and Freq denote the number of on-chip MAC units and the operating frequency, respectively. For instance, on a Stratix-V-A7 FPGA which has 256 DSPs, the highest attainable inference throughput is 204.8 GOP/s under a frequency of 200 MHz (each DSP can perform two 16/8-bit fixed-point MACs). Unless customized MAC units are built by using logic resources, which is generally inefficient both in terms of speed and hardware cost on FPGA device, the performance of the accelerator could not surpass this computational roof. Consequently, the highest throughput achieved by state-of-the-art SDConv designs of [10] and [11] on the same device are 134 and 120 GOP/s, respectively. Both designs have used up all the on-chip DSP blocks.

The second category of designs [16]–[19] implement convolution computation in the *frequency domain* (referred to as FDConv). By transforming the data into frequency domain representation, the sliding window operation of SDConv turns into a inner product operation, which significantly reduces the number of MAC operations required for convolution. As shown in Fig. 1, the FDConv-based accelerator can raise the computational roof over SDConv-based design by a factor of R_{mac}, where R_{mac} is the reduction rate in MAC operation. Very recently, Zeng *et al.* [19] presented a highly optimized FDConv algorithm which saves

Manuscript received August 1, 2019; revised October 21, 2019; accepted December 28, 2019. Date of publication January 20, 2020; date of current version November 20, 2020. This work was supported in part by the NNSF of China under Grant 61574013, in part by the CSC Scholarship under Grant 201807095007, in part by the BJTU-Kuaishou under Research Grant DJHL-20190202-006, in part by the BJTU-ZTE under Research Grant 2018ZTE05-01, and in part by the Beijing Natural Science Foundation under Grant 4202063. This article was recommended by Associate Editor L. Benini. *(Corresponding author: Dong Wang.)*

Dong Wang is with the School of Computer and Information Technology, Beijing Key Laboratory of Advanced Information Science and Network Technology, Beijing 100044, China (e-mail: wangdong@bjtu.edu.cn).

Ke Xu and Jingning Guo are with the Institute of Information Science, Beijing Jiaotong University, Beijing 100044, China.

Soheil Ghiasi is with the Department of Electrical and Computer Engineering, University of California at Davis, Davis, CA 95616 USA (e-mail: ghiasi@ucdavis.edu).

Digital Object Identifier 10.1109/TCAD.2020.2968023

0278-0070 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

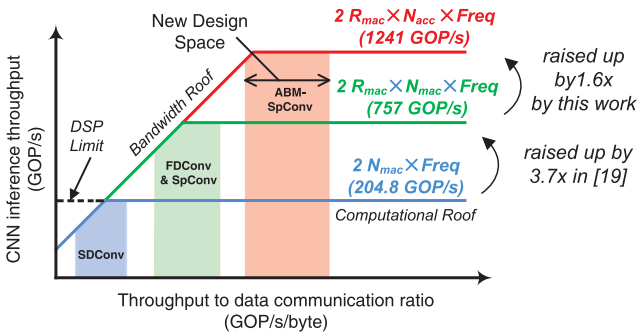

Fig. 1. Design space of different CNN inference accelerators on Stratix-V-A7, which illustrates updated data compared to those reported in [24]. Note that the proposed technique significantly relaxes the DSP limit.

TABLE I
COMPARISON OF THE LOGIC AND DSP UTILIZATIONS OF STATE-OF-THE-ART ACCELERATORS

	[9]	[10]	[15]	[16]	[19]
Logic Usage	18%	52%	52%	58%	46%
DSP Usage	98%	100%	100%	97%	100%

up to 73% of the MAC operation and results in a theoretical speedup of 3.7× in peak performance. The inference throughput achieved by the final design on a Intel HARP platform is 669.1 GOP/s, which is very close to this roof.

The last type of designs adopt *sparse convolution* (SpConv) scheme which saves MAC operation by directly pruning the CNN model [20]–[23]. Unimportant weights (parameters) are forced to zero during the training or fine-tuning stages so that they no longer contribute to any computational workload and memory bandwidth during inference computation. The design space of SpConv-based accelerators share a similar theoretical computational roof of $2 \times R_{\text{mac}} \times N_{\text{mac}} \times \text{Freq}$ with FDConv-based ones.[1] The very latest study of [23] presents a hardware accelerator which implements an element-matrix multiplication-based SpConv algorithm and achieves 2.4 ×–12.9 × speedup over SDConv-based designs on a Xilinx ZCU102 FPGA.

From an architecture view, existing FPGA accelerators tend to adopt a similar underlying architecture based on MAC arrays. The practical consequence is that the on-chip DSPs are often exhausted, while leaving other computational resource under-utilized as shown in Table I. However, in real-world applications such as robotic [6], surveillance system [7], and autonomous vehicle [8], the CNN inference accelerator has often to be used together with other functional units, which may also require extensive use of the DSPs. The overall system performance can be heavily degraded due to the competing demands for DSP resource when all units were integrated on a single device, especially when low-end embedded FPGAs with limited DSP blocks were considered.

In this article, we present a new hardware architecture that delivers superior sparse CNN inference performance and, at the same time, greatly reduces the DSP consumption over existing MAC-based FPGA accelerators. Considering the fact that accumulator is more efficient than MAC both in terms of speed and resource utilization, our approach transforms the accelerator design space in which the computational roof is bound by accumulator resource rather than MAC. The insight enabling this idea is that quantized CNN models only have a fixed number of unique quantization values (for instance, 16 values for a 4-bit fixed-point number), so many multiplications performed in convolution are redundant and can be avoided by weight-sharing (factorization). To this end, we first develop a new SpConv approach which performs the multiplication and accumulation operations in decoupled computational stages with different arithmetic intensity. Then, we design a heterogeneous hardware architecture comprised of a "big" accumulator array and a "small" multiplier array on which the two stages of convolution can be independently executed. This new approach transforms the design space of FPGA-based CNN accelerators to one with a raised computational roof of $2 \times R_{\text{mac}} \times N_{\text{acc}} \times \text{Freq}$, where N_{acc} denotes the number of accumulators used and N_{acc} can be much larger than N_{mac} on most FPGA devices as shown in Fig. 1.

The idea of performing design space transformation was first presented in our previous study of [24]. However, the work have left a few design issues unaddressed that have prevented the final design from achieving the highest efficiency in on-chip DSP resource utilization.

1) The directly quantized neural network model has unbalanced accumulation/multiplication arithmetic intensity among different layers which have caused a mismatch of the new convolution algorithm and hardware architecture for certain layers.
2) The dataflow of the algorithm was not properly modeled and loop tiling technique was not adopted, both of which caused a low utilization of the pipeline circuits and external memory bandwidth.
3) The proposed architecture was only implemented on a Stratix-V device and the effectiveness of the proposed scheme for low-cost embedded FPGAs was not verified.

Moreover, only two CNN models were mapped, whereas state-of-the-art neural networks, such as ResNet or GoogleNet with more complex network architectures, were not tested. Therefore, the versatility of the proposed scheme remained unclear. We address these issues in this extended work and present a further optimized design with improved performance and hardware resource utilization.

In summary, the contributions of this article are as follows.

1) We propose a novel SpConv scheme, namely, accumulate-before-multiply SpConv (ABM-SpConv), which utilizes weight-sharing in quantized CNN model to obtain higher arithmetic intensity for accumulation over multiplication so that the computational roof of the corresponding accelerator design space can be transformed as accumulator-bound. An optimized dataflow that decouples the accumulation and multiplication in independent computation stages and efficiently exploits both loop unrolling and tiling techniques in multiple

[1]Although, extra sparsity of the input feature map can be utilized to further reduce MAC operations for SpConv-based designs, the number heavily depends on the input data set, which makes a quantitative measure of the performance boost infeasible. In this article, we only consider SpConv schemes that exploit sparsity in the CNN model.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

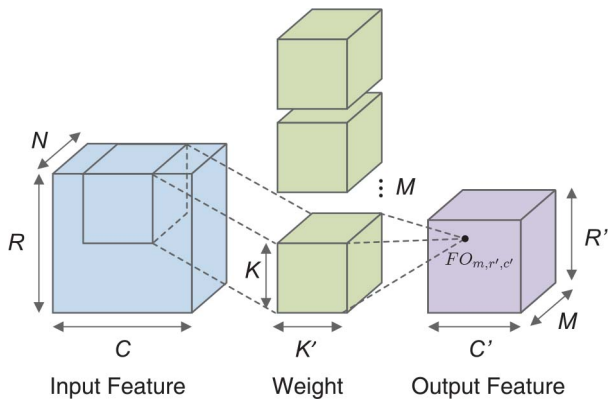

Fig. 2. Illustration of the convolution computation.

dimensions of the convolution computation to improve throughput is also modeled.

2) Based on the optimized dataflow, we propose a CNN inference accelerator architecture for FPGA, which consists of heterogeneous arrays of accumulators and multipliers to match the distinct arithmetic intensity of the two operations. Several optimization schemes, including semisynchronous parallel processing, parallel line-buffer-based on-chip data caching and index-based weight encoding and decoding, are also developed to ensure highly efficient data-path utilization and low external memory bandwidth requirement.
3) An end-to-end design space exploration (DSE) flow was introduced, and key design steps for preprocessing of the CNN model (pruning, clustering, and quantization) and finding the optimal hardware parameters are developed. We have conducted comprehensive evaluation of the performance and resource utilization of the proposed accelerator on three difference FPGA devices, including Cyclone-V, Stratix-V, and Arria-10, which covered a wide range of target application scenario from embedded settings to high-performance computing. The state-of-the-art CNN models, including AlexNet, VGG, and ResNet, were also tested as performance benchmarks to evaluate the versatility of the proposed scheme.
4) We demonstrate that our approach achieves 1.5× boost in inference throughput and 4× reduction in DSP usage compared to the best design which implements FDConv on a Stratix-V GXA7 FPGA. When compared to state-of-the-art SpConv-based accelerator, the proposed design has a 2.5× advantage in raw arithmetic performance and 8.4× saving in DSPs on the Arria-10 device.

II. Preliminaries

In this section, we define a few terms that are used in the following discussions. CNNs are generally composed of multiple functional layers [2], each of which performs a certain type of arithmetic operation on the input image or feature map. Among all the layers, convolution layer is the most compute-intensive one, which accounts for most of the workload in CNN inference computation. Convolution is comprised of iterative 3-D MAC operations as follows:

$$FO_{m,r',c'} = \sum_{n=1}^{N}\sum_{k=1}^{K}\sum_{k'=1}^{K'} FI_{n,r'\cdot S+k,c'\cdot S+k'} \cdot W_{m,n,k,k'} \quad (1)$$

where $FI_{n,r,c}$, $FO_{m,r',c'}$, and $W_{m,n,k,k'}$ denote the input and output feature maps and the weight matrices, which are of the size $N \times R \times C$, $M \times R' \times C'$, and $M \times N \times K \times K'$, respectively. S denotes the stride of the convolution sliding window. The dimensional parameters of the convolution layer are captured in Fig. 2. As the green cube shows, a *convolution kernel* (*or filter*) is defined as the sets of weight with a volume size of $N \times K \times K'$, and there are M kernels in total in each layer. *Channels* refer to the feature map matrices of size $R \times C$ or $R' \times C'$. For FC computation, (1) can be reused by setting $R = 1, C = 1$, and $K = 1, K' = 1$, which becomes a 1-D inner-product operation. Table II gives an example of the dimensional parameters and the computational workload in terms of the number of operations (#*OP*) for each layer of the VGG16 [1] model that has been widely used as a performance benchmark in the literature. It can be observed from the eighth column that a direct implementation of (1) without any algorithm-level optimization, i.e., using the SDConv scheme, requires a computational workload of 30 GOPs to conduct one time of inference computation. In contrast, after adopting the FDConv or SpConv schemes, the computational workload of the CNN model is reduced by more than 3×, which can significantly boost the performance of the implemented hardware accelerator. However, the algorithm optimizations conducted by traditional FDConv and SpConv approaches are still based on the MAC operation, which often leads to an unbalanced hardware resource utilization between FPGA logic and DSP blocks as shown in Table I. In this article, we treat the accumulation and multiplication of the MAC operation independently and propose a new SpConv algorithm that can reduce the workload of the two operations separately to achieve balanced resource utilization. We present the new convolution scheme in the following section.

III. ABM-Sparse Convolution

In this section, we introduce the ABM-SpConv scheme. The key idea behind this scheme is to decouple the accumulate and multiply operations in two separated stages so that the arithmetic intensity of multiplication can be reduced independently from accumulation by sharing unique weight quantization values. A direct benefit is that it significantly relaxes the resource demand for multipliers (DSP units) when implemented in hardware (FPGA).

A. ABM-SpConv Scheme

Assuming that weight W is quantized in fixed-point format with q-bit precision, then there only exists Q ($Q \leq 2^q$) unique quantization values for W. By denoting these unique fixed-point weights as $\bar{W}_p$, where $p = 1, \ldots, Q$, (1) can be

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

```
    _buffer_ Acc[Q];
(a) for m = 1 to M
(b)   for r' = 1 to R'
(c)     for c' = 1 to C'
(d)       for p = 1 to Q_m                          Accumulation
(e)         for i = 1 to w_p {                        Stage
              Acc[p] += FI_p[i];
            }
          }
(f)       for p = 1 to Q_m {                        Multiplication
              FO[m][r'][c'] += Acc[p] * W̄_p[m];        Stage
          }
```

Fig. 3. ABM-SpConv convolution dataflow illustrated as a loop nest. Accumulation and multiplication are decoupled in independent stages with different workload.

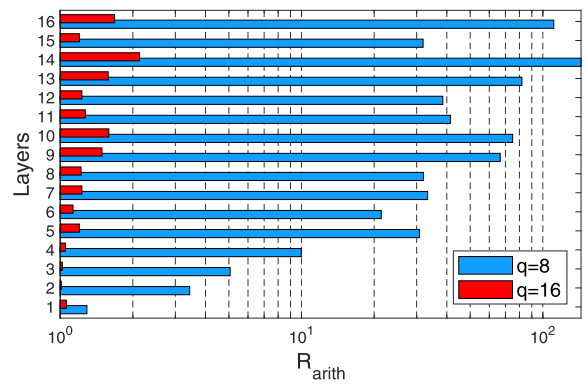

Fig. 4. Arithmetic intensity ratio of accumulation to multiplication of a pruned VGG16 models [28] when quantized in 8- and 16-bit fix-point format, respectively.

transformed as follows by factorization:

$$FO_{m,r',c'} = \sum_{i=1}^{w_1} FI_1[i] \cdot \bar{W}_1 + \cdots + \sum_{i=1}^{w_Q} FI_Q[i] \cdot \bar{W}_Q = \sum_{p=1}^{Q} \left(\bar{W}_p \cdot \sum_{i=1}^{w_p} FI_p[i] \right) \tag{2}$$

where $FI_p[i]$ represents the input feature map pixels that are multiplied by the same unique weight value $\bar{W}_p$ within a convolution kernel, and w_p denotes the specific number of the feature map pixels that corresponds to that $\bar{W}_p$. Note that fixed-point arithmetic is conducted in (2), therefore, to ensure the correctness of mathematical derivation, we assume that all the intermediate results were kept in full precision and no rounding or overflow errors were introduced. However, we have found that, in the case of CNN inference computation, it is possible to reduce the bit-width of the data-path and save hardware resource without sacrificing the inference accuracy. This design issue is addressed in detail in Section IV-B.

When the CNN model is properly quantized, the accumulate operation in (2) can have a much higher arithmetic intensity than that of multiplication. To utilize this feature, we propose a convolution dataflow, as shown in Fig. 3, in which the accumulation of input feature pixels [loops (d) and (e)] and multiplication with the unique quantized weight value [loop (f)] are carried out in two stages. The dataflow utilizes an on-chip memory Acc[Q] to buffer the partial convolution results [i.e., outputs of the accumulation loops (d) and (e)], which enables us to map the two distinct computational stage onto two independent hardware units with different amount of computational resources.

In addition, the zero-valued weights ($\bar{W}_p = 0$) can be discarded during weight encoding, therefore, the proposed convolution scheme can take full advantage of sparsity in pruned CNN model to further reduce overall computational workload and save memory bandwidth spent on weight transferring as traditional SpConv schemes do.

B. Quantization Effect on Arithmetic Intensity

We define R_{arith} as the arithmetic intensity ratio of accumulate to multiply operations involved in the ABM-SpConv approach. For a specific layer of a pruned CNN model, R_{arith} can be quantitatively evaluated by

$$R_{\text{arith}} = \frac{K \cdot K' \cdot N \cdot M \cdot R_{\text{prun}}}{\sum_{m=1}^{M} Q_m} \tag{3}$$

where R_{prun} represents the pruning rate of the convolution/FC layers. The exact number of unique weight values is usually inconsistent among different convolution channels, so we introduce Q_m to denote the exact number of $\bar{W}_p$ in the mth convolution channel. Then, $\sum_{m=1}^{M} Q_m$ equals the total number of multiplications that needs be conducted in this layer. After analyzing several widely used CNN models, including AlexNet, VGG, and ResNet, it was found that Q_m was highly determined by the bit-width configured during weight quantization. To illustrate the effect of quantization precision q on the value of R_{arith}, we compare the arithmetic intensity ratio for all layers of a pruned VGG16 model in Fig. 4. The model was first pruned by using the pruning scheme introduced by Han *et al.* [28] and then quantized in fixed-point format with 8-bit and 16-bit word length, respectively. Model quantization adopts the flow presented by Gysel *et al.* [26] because it does not require extra processing of the weight and excessive retraining of the model, which ensures that the pruning rate R_{prun} of the baseline weight remains the same between both quantization settings. The detailed #*OP* are also reported in the last three columns of Table II.

It can be observed from Fig. 4 that 16-bit quantization does not significantly reduce the arithmetic intensity of multiplication as the resulting R_{arith}'s are close to 1. In contrast, 8-bit quantization achieves a much higher R_{arith} for most of the layers. By comparing the data reported in the last two columns of Table II, one could see that the main cause is the differences in #*OP* of multiplication between 16- and 8-bit quantized models. Generally speaking, a shorter bit-width q used in quantization sets a lower bound 2^q for Q_m, which eventually results in fewer number of unique weight values. Moreover, it could also be found that, even for 8-bit quantization scheme, the #*OP* of accumulation is very close to that of multiplication in the first two layers due to the small kernel size ($N \times K \times K'$), which has lowered the frequency of weight sharing. In [24], these small values of R_{arith} have prevented us from achieving the

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF THE COMPUTATIONAL WORKLOAD (#OP) REQUIRED BY DIFFERENT CONVOLUTION APPROACHES FOR A PRUNED VGG16 MODEL[28]

	Model Parameters and Pruning Rate						#OP (10^6)					
										ABM-SpConv		
Layer	C	R	N	$K \times K'$	M	R_{prun}	SDConv	FDConv[19]	SpConv[28]	Acc.	Mult. ($q = 16$)	Mult. ($q = 8$)
CONV1_1	224	224	3	3×3	64	42%	173	50.3	101	50.3	47.4	38.8
CONV1_2	224	224	64	3×3	64	78%	3,699	1,073	814	407	403	119
CONV2_1	112	112	64	3×3	128	66%	1,849	536	629	314	307	62.7
CONV2_2	112	112	128	3×3	128	64%	3,699	1,073	1,368	684	651	68.5
CONV3_1	56	56	128	3×3	256	47%	1,849	536	980	490	408	15.9
CONV3_2	56	56	256	3×3	256	76%	3,699	1,073	888	444	392	20.7
CONV3_3	56	56	256	3×3	256	58%	3,699	1,073	1,554	777	631	23.3
CONV4_1	28	28	256	3×3	512	68%	1,849	536	592	296	242	9.25
CONV4_2	28	28	512	3×3	512	73%	3,699	1,073	998	499	335	7.49
CONV4_3	28	28	512	3×3	512	66%	3,699	1,073	1,257	629	395	8.38
CONV5_1	14	14	512	3×3	512	65%	924	268	324	162	127	3.92
CONV5_2	14	14	512	3×3	512	71%	924	268	268	134	109	3.48
CONV5_3	14	14	512	3×3	512	64%	924	268	333	166	105	2.19
FC6	1	1	25088	1×1	4096	96%	205	205	8.23	4.11	1.93	0.028
FC7	1	1	4096	1×1	4096	96%	33.6	33.6	1.34	0.67	0.56	0.021
FC8	1	1	4096	1×1	1000	77%	8.2	8.2	1.88	0.94	0.56	0.008
Total							30,941	9,156	10,109	5,058	4,155	383

best result in reducing the number of multipliers (DSP blocks) used in the final design. To address this issue, in this article, we propose to perform weight clustering on those layers that have small values of R_{arith}. The weight clustering operation can add more restricted constrains to manually control the desired value of Q_m and consequently reduce the total number of unique values of the quantized weight. The detailed quantization and weight clustering flow is discussed in Section V-B. Finally, considering that 8-bit quantization has already been proven being sufficient to maintain the same inference accuracy for state-of-the-art CNNs [26], [28], [30], the proposed hardware accelerator will be designed and explored based on 8-bit quantized CNN models.

C. Exploring Parallelism and Data Sharing

In this article, we only consider unstructured model pruning schemes [31], in which unimportant weights are trimmed according to their values rather than geometric information and thus the nonzero weights of the pruned model are distributed randomly in the convolution kernel. Consequently, the accumulation loops of (d) and (e) presented in Fig. 3 have variable loop bounds, which makes intra-kernel parallelization through direct loop unrolling infeasible. Therefore, we introduce batched input to the original dataflow and propose a throughput-optimized ABM-SpConv dataflow that explores parallelism on the outside loops (a) and (b) as shown in Fig. 5, in which N_{ba} denotes the batch size of the input feature maps. As illustrated by the *parallel-for*, all N_{ba} input feature maps are convolved by the same kernel simultaneously. Loops (a) and (b) are unrolled by the factor of N_m and N_r, respectively, and thus parallel convolution are also conducted in different channels of the output feature map and on multiple sliding windows along the column direction of the input feature map. For convolutional layers, the optimized dataflow can boost the inference throughput by a maximum rate of $N_{ba} \times N_r \times N_m$ times. For FC layers, the size of the input feature map shrinks to $1 \times 1 \times N$, and consequently the throughput gain scales down to $N_{ba} \times N_m$. Since FC layers only contribute to a small portion of the total computational workload, the overall performance improvement is still close to that of the convolutional layer.

```
    _buffer_ FT_Buf[N_m][D_f];
    _buffer_ WT_Buf[N_m][D_w];
    _buffer_ Q_Buf[N_m][Q];
    _buffer_ Acc[N_m][N_r][N_ba][Q];
(a) for m = 1 to ⌈M/N_m⌉
(b)   for r' = 1 to ⌈R'/N_r⌉
(c)     for c' = 1 to ⌈C'/N_c⌉
(d)       for i = 1 to N_m, n = 1 to N, k = 1 to K, k'= 1 to K'
(e)         for p = 1 to Q_{N_m*m+i}
              WT_Buf[i][p] = (n,k,k') when W(N_m*m+I,n,k,k')=W̄_p
(f)       for k = 1 to N_ba
(g)         for s = 1 to D_f and r ∈[r', r'+N_r*S+K) and c ∈[c', c'+N_c*S+K')
                FT_Buf[k][s] = FI_{n,r,c}[k];
(h)       for t = 1 to N_c
(i)         parallel_for [i = 1 to N_m] x [j = 1 to N_r] x [k = 1 to N_ba]
(j)           for p = 1 to Q_{N_m*m+i}   {
(k)             for w = 1 to w_p {
                  Acc[i][j][k][p] += FT_Buf[k][FIND_IDX(WT_buf[i][p*w_p+w])];
                }
              }
(l)           for p = 1 to Q_{N_m*m+i}   {
                FO[k][m*N_m+i][r'*N_r+j][c'*N_c+t] += Acc[i][j][k][p]*Q_Buf[i][p];
              }
```

Fig. 5. Throughput-optimized convolution dataflow. Loops (d)–(g) initialize the local buffers at the start of each output iteration. The parallel-for loop represents the parallel convolutions that are executed in the output dimension m and r and among all the batched input feature maps.

As defined in Fig. 5, two local buffers are introduced to cache the input feature map pixels (in *FT_BUF*) and the logic coordinates of the nonzero weights (in *WT_BUF*), both of which are shared among consecutive and parallel convolution computations. Another small table *Q_BUF* is used to store the unique weight quantization values. In the optimized dataflow, the same weights are shared among the $N_r \times N_{ba}$ parallel convolutions conducted in the output dimension r' and on the batched inputs, which can reduce the bandwidth spent on weight transferring by $N_r \times N_{ba}$ times compared to the unoptimized dataflow. Function *FIND_IDX*() calculates and returns

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

the physical buffer addresses of the feature map pixels $FI_p[i]$ that corresponds to the same $\bar{W}_p$ according to the stored logic coordinates of the weights. Because sliding windows along the dimension r' and c' are overlapped with adjacent ones with a fixed stride, a large portion of the buffered feature map data can also be reused. Therefore, we further tile loop (c) and then the feature map cached in *FT_BUF* can be shared by the N_r parallel convolutions and reused by the consecutive convolutions repeated along the dimension c'. However, the exact saving on bandwidth depends on the specific structure of the feature map buffers. Detailed on-chip buffer design and implementation of the function *FIND_IDX*() will be elaborated in Section IV-B.

IV. Hardware Architecture

A. Design Challenges

The new convolution scheme introduces the following hardware design challenges.

1) The accumulate and multiply operations exhibit different arithmetic intensity and are carried out in two distinct computation stages. Mapping both operations on existing homogeneous hardware architecture like [14] and [19] will cause low utilization of computational resources.
2) The irregular sparsity pattern among different convolution kernels introduces imbalanced workload, which prevent us from achieving full parallelism of convolution on an accelerator architecture consisting of tightly synchronized executed processing elements, such as MAC array.
3) The algorithm also requires random memory access to the feature map data in memory accordingly to the irregular locations of the nonzero weights, which will degrade the efficiency of external memory bandwidth and require complicated data flow control in data path design as in [23].

In the following section, we discuss our proposed hardware architecture, which addresses the aforementioned design challenges.

B. Architecture Design

Fig. 6(a) shows the overall architecture of the proposed FPGA accelerator that can efficiently implement the optimized ABM-SpConv dataflow of Fig. 5. The accelerator consists of a task scheduling unit, a fetch/store unit, multiple convolution units (CUs), and a miscellaneous (MISC.) function unit. The task scheduler periodically detects the status of each CU and, whenever there is an idle CU, it launches a new *computation task* on that CU and send the fetch/store unit a command to load required feature map and weight data from external memory to the local buffers. The MISC. unit implements additional layer functions, such as batch normalization, Eltwise, etc.

1) Convolution Unit: To address the first design issue, we propose a heterogeneous CU architecture which consists of two independent arrays of accumulators and multipliers as shown in Fig. 6(b). A total number of $N_m \times N_r \times N_{ba}$ parallel

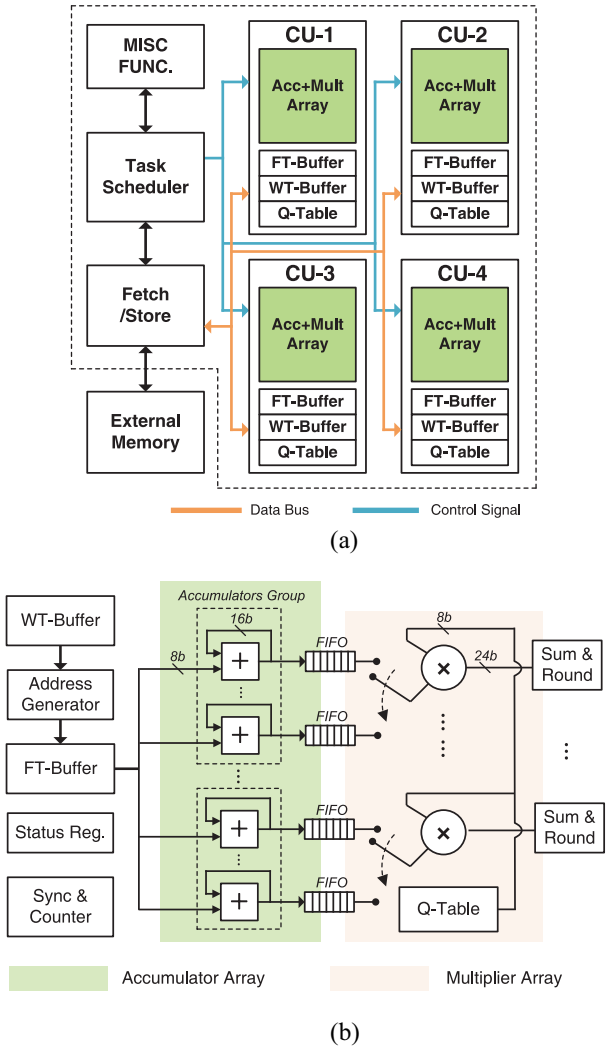

Fig. 6. Proposed accelerator architecture. (a) High-level architecture. (b) Architecture of the CU.

accumulators is designed in the accumulator array supporting all the parallel computation tasks defined by the parallel-loop in Fig. 5. In every cycle, the accumulator adds the input feature map pixel $FI_p[i]$ to the feedback partial result and increments the companion counter by 1. As soon as all the feature map pixels that share the same $\bar{W}_p$ have been accumulated, i.e., the counter reaches the bound w_p, the partial convolution result is send into following FIFO.

The accumulators are further divided in groups and each group contains N_g accumulators sharing a single multiplier in the pipeline. During convolution, the multiplier periodically chooses the outputs of upstream FIFOs as its input operand in a round-Robin manner. The partial convolution results are then multiplied by the corresponding $\bar{W}_p$ and the final products are send to the sum/round logic for final rounding. For a given CNN model, with a proper setting of N_g that matches to the arithmetic intensity ratio of R_{arith}, the proposed two-stage convolution dataflow can be efficiently pipelined on this heterogeneous accumulator+multiplier array structure.

As mentioned in Section III-A, it is mathematically required that all intermediate accumulation results must be kept with

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

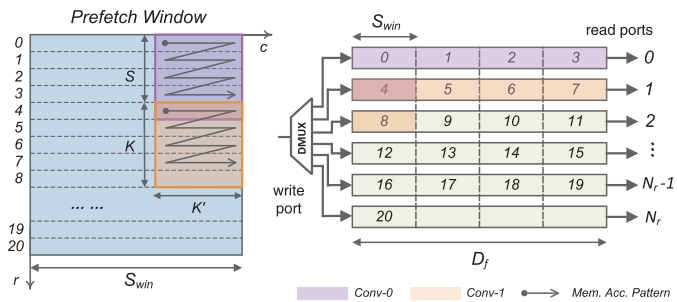

Fig. 7. Feature map data prefetching strategy and internal structure of the feature map buffer. The example assumes a convolution kernel of size $K = K' = 5$, $N = 1$, and $S = 4$, and architecture configuration of $N_r = 5$. Adjacent convolutions (such as Conv-0 and Conv-1) have a fixed stride of 4, therefore, in each line-buffer, only four lines of feature map data are stored.

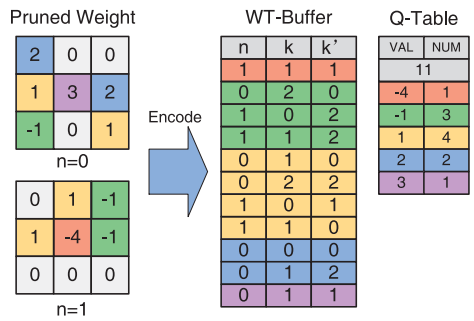

Fig. 8. Proposed sparse weight encoding scheme. A practical example for kernel size of $M = 1, N = 2$, and $K' = K = 3$ is illustrated. Weighs are quantized in 3-bit 2s complement.

full precision to guarantee the correctness of (2). For instance, the accumulator must have a minimum width of 21 bits to avoid any possible overflows when the convolution kernel is of the size $3 \times 3 \times 512 = 4608$ and the feature map data is quantized in 8 bits. Reducing the accumulator bit-width may introduce overflow error which will cause accuracy drop in CNN inference. However, in pruned models, a considerable amount of weights have already been set to zero, which significantly reduced the chance for overflow. On the other hand, studies, such as [26] and [27], have also shown that neural networks are robust to computational errors. Therefore, in the final design, we implemented the accumulators with different precisions from 32 to 16 bits and measured the accuracy drop for each precision configuration. The results have shown that a 16-bit accumulator was sufficient to maintain the same inference accuracy for all the models we have tested.

2) On-Chip Buffer: As discussed in Section III-C, N_r convolutions with overlapped sliding windows along the input dimension r are conducted in parallel in the optimized ABM-SpConv dataflow. To enable simultaneous access of the feature map data by parallel convolution tasks and maximize data sharing, we propose a single-input-multiple-output line-buffer-based structure to implement the feature map buffer (FT-Buffer). As illustrated in Fig. 7, the FT-Buffer consists of $N_r + 1$ two-port RAMs, which have $N_r + 1$ independent read ports and a single write port. As the sliding windows of adjacent parallel convolutions have a common fixed stride of S lines within the prefetch window, which means that the reading addresses also have a fixed offset when mapped into a linear address space, we utilize this special memory access pattern and store the prefetched feature map in a line-by-line manner with an offset of S lines to avoid memory access collisions between adjacent convolution tasks. The prefetch window is of the size S_{win} in dimension c. So the depth D_f of the feature map buffer must be larger than $S_{\text{win}} \times S \times N$ for all layers. Feature map data within a batch are also merged into vector data to improve utilization of the external memory bandwidth and the width of each line buffer is of $8 \times N_{ba}$ bits. Theoretically, a batch size of 64 can saturate the 512-bit DDR memory interface on FPGA.

3) Weight Encoding: In previous SpConv-based accelerators, such as [20] and [23], both weights and corresponding coordinates of each nonzero weight were encoded in a one-to-one correspondence way. The extra bits spent on coordinate information would eliminate the benefit of reduced memory footprint and external bandwidth. In this article, we propose to encode the coordinates along with only a small amount of unique quantization values to overcome the drawback of previous encoding schemes. As illustrated in Fig. 8, the proposed scheme repeatedly finds the coordinates (n, k, k') that corresponds to each of the unique quantization value $\bar{W}_p$ and stores them in WT-Buffer. The searching follows a zig-zag order starting from the top left corner of the weight matrix. The coordinates that corresponds to the same $\bar{W}_p$ are located in adjacent positions (illustrated in the same color) and the coordinate groups are arranged according to the order of $\bar{W}_p$ to alleviate the controlling overhead. Another small buffer (Q-Table) is designed to store extra information, including the unique quantization value (VAL), the corresponding number (NUM) of occurrence of coordinates, and the total number of occurrence of the encoded weights, which are used by the loop counters and multipliers in the CU. During decoding, the coordinates are read sequentially by the address generator and mapped onto the feature map domain. Both WT-Buffer and Q-Table are designed with 16-bit width. Table III compares the sizes of the original and encoded weights. It can be seen that the proposed scheme can reduce the memory footprints by 20×.

4) Address Generator: A dedicated Address Generator is designed to decode the weights on-the-fly and load continuous data stream from the feature map buffer. As discussed in Section III-C, the function *FIND_IDX*() maps the logical coordinates of (n, k, k') to the feature map domain to generate the physical addresses of the FT-Buffer. As the feature map data are organized in the form of prefetch windows as illustrated in Fig. 7, the address calculation is performed according to the following formula:

$$\text{Addr}_{\text{FT-Buf}} = S_{\text{win}} \cdot S \cdot n + \text{MOD}(k, S) + S \cdot t + k' \quad (4)$$

where t is a counter that records the index number of the sequential convolutions to be conducted along dimension c' as defined in Fig. 5. MOD represents a modulo function. Since in most CNNs, the stride S is normally small and there are only a few frequently used values, such as 1, 2, and 4, we

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

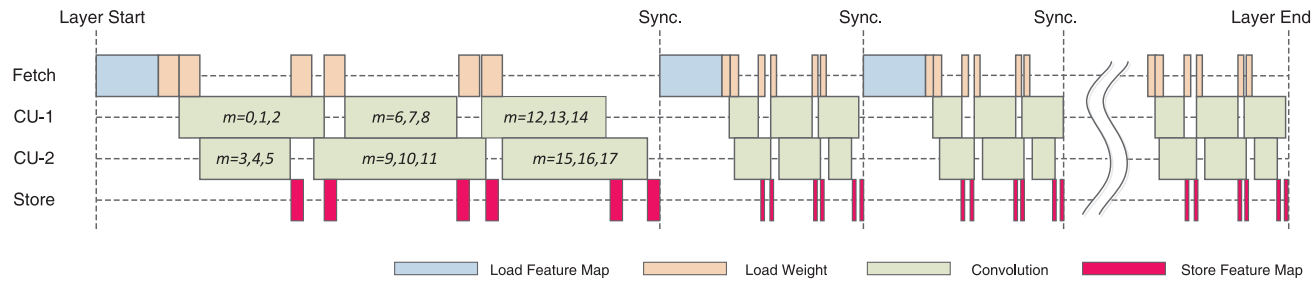

Fig. 9. Scheduling of the data fetch/store and convolution tasks. In this example, we assume the accelerator has two CUs and encoded weight of three convolution kernels are loaded together into the WT-Buffer each time the task scheduler launches a convolution task on a CU. Note that the timing scale used in the figure does not represent the actual length of time. In the final design, the data fetching time is much shorter than that of convolution computation.

have implemented the modulo function as bit-shifting circuits in the final design to save hardware resources.

5) Task Scheduling: As shown in Fig. 6(b), each CU has its own loop counter so that it can run independently executing convolution tasks with varying workloads. Fig. 9 shows an example of how data fetch/store and convolution tasks are scheduled by the proposed accelerator. At the start of each layer convolution, the fetch/store unit first loads the feature map prefetch window and broadcasts the data to the local buffers in all CUs. Then, the task scheduler finds an idle CU (i.e., CU-1) and send a command to both fetch/store unit and CU triggering a new convolution task. The fetch/store unit quickly loads the encoded weight in the WT-Buffer and Q-Table and the CU immediately starts to work. Synchronization of the CUs is infrequently conducted, only when the FT-buffer needs to be updated. Therefore, a larger prefetch window will generally reduce the frequency of synchronization, which improves the efficiency of the CUs but consumes more on-chip memory resources. The optimal size of the feature map buffer will be determined based on tradeoffs between hardware cost, bandwidth utilization and CU efficiency (see Section VI).

6) Design Parameters: In summary, the proposed architecture can be configured by the following design parameters to achieve flexible performance and hardware cost.

1) N_m—the number of the asynchronous CUs; N_r—the maximum number of parallel convolution computations along the input dimension r that can be mapped on the accumulator array; N_{ba}—the input batch size; and N_g—the ratio of the number of accumulators to multipliers. Totally, the accelerator consumes $N_m \times N_r \times N_{ba}$ accumulators and $(N_m \times N_r \times N_{ba})/N_g$ multipliers.
2) D_f, D_w, and D_q—the depth of the FT-Buffer, WT-Buffer, and Q-Table local buffers, respectively.

V. Design Space Exploration

DSE plays an important role in FPGA-based CNN accelerator design. In this section, we first define the mathematical models for performance and hardware cost evaluation and then, present an end-to-end accelerator DSE flow and explain how these models are applied in finding the optimal design parameters.

A. Performance and Resource Estimation Models

The *performance model*, *bandwidth model*, and *resource requirement model* used in the proposed DSE flow are defined as follows.

1) Performance Model: The average execution time for the lth convolution/FC layer is estimated by

$$T_l = \frac{\#OP_l}{N_{ba} \cdot N_m} \cdot \frac{\text{ceil}\left[R'/N_r\right]}{R'} \cdot \frac{1}{\text{Freq}} \tag{5}$$

where $\#OP_l$ is the total number of accumulations performed in the lth layer. Assuming the execution time of other functional layers is marginal to the total runtime, the overall throughput (image/s) can be estimated by

$$P_{\text{total}} = \frac{1}{\sum T_l}. \tag{6}$$

2) Bandwidth Model: In the lth layer, the whole input feature map is processed after $G_l^r \cdot G_l^c$ times of prefetching, where G_l^c and G_l^r equal

$$G_l^c = \text{ceil}\left[\frac{C'}{\text{floor}\left[(S_{\text{win}} - K)/S\right] + 1}\right] \tag{7}$$

$$G_l^r = \text{ceil}\left[\frac{R'}{N_r}\right]. \tag{8}$$

Consequently, the total amount (byte) of feature map data (quantized in 8-bit precision) that are transmitted from external memory per image inference is

$$H_f = \sum_l \left(G_l^c \cdot G_l^r \cdot S_{\text{win}} \cdot \left[(N_r - 1) \cdot S + K'\right] \cdot N\right). \tag{9}$$

Normally, the amount of the encoded unique quantization values is marginal to the encoded coordinates. Therefore, we use the total number of transferred encoded coordinates (16-bit word-length) to estimate the average bandwidth per image

$$H_w = \sum_l \left[2 \cdot G_l^c \cdot G_l^r \cdot M \cdot N \cdot K \cdot K' \cdot (1 - R_l)\right]/N_{ba} \tag{10}$$

where R_l denotes the pruning rate of the lth layer. In total, the average external memory bandwidth can be estimated by

$$H_{\text{total}} = \frac{H_f + H_w}{\sum T_l}. \tag{11}$$

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

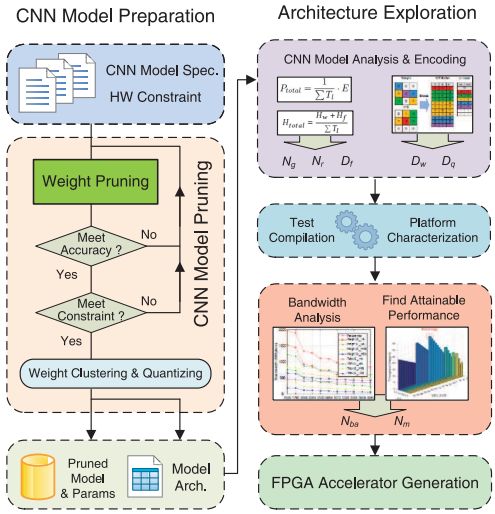

Fig. 10. Overview of the DSE flow.

3) Resource Requirement Model: As has been discussed in Section IV-B, the proposed accelerator consumes a total number of $N_{ba} \times N_r \times N_m$ accumulators and $(N_{ba} \times N_r \times N_m)/N_g$ multipliers. The accumulators and multipliers are organized as N_m arrays located separately in the CUs. For each accumulator array, one can estimate that the logic resource scales linearly with $N_{ba} \times N_r$, and therefore, the resource model can be defined by

$$C_{\text{logic}} = C_0 + (C_1 + C_2 \cdot N_{ba} \cdot N_r) \cdot N_m \tag{12}$$

where C_2 denotes the average logic count spent on one accumulator while C_0 and C_1 represent the logic resource consumed by other functional units in the accelerator. Similarly, the DSP resource model is define by

$$C_{\text{dsp}} = C_3 + \left[C_4 + (N_{ba} \cdot N_r)/N_g\right] \cdot N_m. \tag{13}$$

The proposed feature map line-buffer uses $N_r + 1$ independent two-port RAMs, each of which is of $8 \times N_{ba}$-bit wide. Moreover, in each CU, the WT-Buffer and Q-Table are of fixed size. Consequently, the overall on-chip memory consumption can be modeled by

$$C_{\text{mem}} = C_5 + (C_6 + C_7 \cdot N_{ba} \cdot N_r) \cdot N_m. \tag{14}$$

As illustrated in Fig. 10, after several round of test compilation, the values of the design constants are calculated by using linear-regression techniques, and then, the obtained models are used in the following exploration steps.

B. DSE Flow

The proposed end-to-end DSE flow is illustrated in Fig. 10. The whole flow is divided into two major stages: 1) CNN model preparation and 2) accelerator architecture exploration.

1) CNN Model Preparation: In this stage, the designer provides the model specifications (types, numbers and interconnections of the functional layers, etc.), the baseline CNN model (32-bit floating-point format) and defines a set of hardware constraints, including the maximum allowed number of nonzero weight in each convolution kernel and the inference accuracy budget for the final design. Then the flow prunes and quantizes the CNN model according to the preset target accuracy. In CNN model pruning, we adopt a heuristic criteria [29] to measure the importance of the weight and propose to accumulate the saliency score over the whole dataset during retraining to obtain a more accurate threshold for weight trimming in each layer than that of [28]. As has been discussed in Section IV-B, the encoded weights are loaded into the WT-Buffer in a kernel-by-kernel manner, so the maximum number of nonzero weight in each convolution kernel must be smaller than D_w. In order to meet this hardware constraint, the pruning flow also automatically counts the occurrence of the nonzero weight in each kernel and round them to the upper nearest power of two by trimming an extra small amount of weights.

After meeting the accuracy budget, the pruned neural network model is quantized with 8-bit precision in dynamic fixed-point format by using the scheme introduced by Gysel *et al.* [26]. As shown in Fig. 4, it was found that, for all the CNN models we have tested, the arithmetic intensity ratios (R_{arith}) were unbalanced among the convolution/FC layers and the R_{arith} of the first two to four layers were usually much lower than the others due to the small kernel sizes. The unbalanced R_{arith} will lead to an underutilization of the proposed heterogeneous array architecture due to a mismatch of N_g with R_{arith}. To solve this problem, we adopt the idea of using K-means clustering from [28] to process the weights for those layers whose R_{arith} does not match the desired N_g. Instead of only having an upper bound of $2^8 = 256$, the exact value of Q_m of each convolution kernel are manually adjusted by weight clustering to the desired value. After weight clustering, the minimum R_{arith} of a CNN model can be generally increased by 2–4 times.

We have integrated the model pruning, clustering and quantization procedures into a single flow by using Pytorch and Python. Three widely used CNN models, including AlexNet, VGG16, and ResNet-50 were pruned and quantized by the proposed DSE flow and the inference accuracy, weight size and the #OP of the baseline and pruned model are compared in Table III.

2) Accelerator Architecture Exploration: After obtaining the quantized CNN model, the flow encodes the weight in a layer-by-layer manner according to the proposed encoding scheme and the buffer sizes D_w and D_q are determined. At the same time, the R_{arith} for each convolution or FC layer is analyzed as reported in Table III and the design parameter N_g is selected to match the minimum arithmetic intensity ratio which guarantees a full utilization of the heterogeneous array structure for all layers. From the *performance model* of (5), it is observed that the speed-up gain introduced by the parallel dataflow over the serial dataflow for a certain layer is affected by the ratio of R/N_r. As shown in Fig. 11, for a specific CNN model, the proposed DSE flow will calculate and compare the average performance gain achieved when configuring the hardware parameter N_r with different values. It can be observed that the benefit decreases rapidly when $N_r > 14$ for all three models. For large values of N_r, the performance boost drops

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

TABLE III
INFERENCE ACCURACY, WEIGHT SIZE, AND #OP OF THE BASELINE CNN MODEL AND THE MODEL PROCESSED BY THE PROPOSED DES FLOW

Model	Pruning Rate		Top-1/5 Accuracy		Weight Size (MByte)		#OP (10^9)			avg. R_{arith}
	Conv.	FC	Baseline	After Pruning	Baseline	Encoded	Baseline	After Pruning	Reduction	
AlexNet	91%	88%	56.55% / 79.09%	56.73% / 79.77%	244	13.2	1.44	0.223	6.5×	9 ∼ 49
VGG16	85%	89%	70.59% / 90.38%	70.05% / 89.81%	552	26.5	30.9	7.48	4.14×	8 ∼ 129
ResNet-50	92%	60%	76.15% / 92.87%	75.70% / 92.88%	102	5.4	7.72	1.12	6.8×	4 ∼ 34

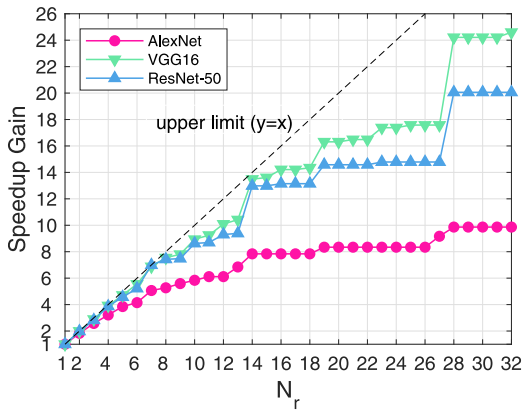

Fig. 11. Exploration for the optimal value of N_r under the working frequency of 200 MHz.

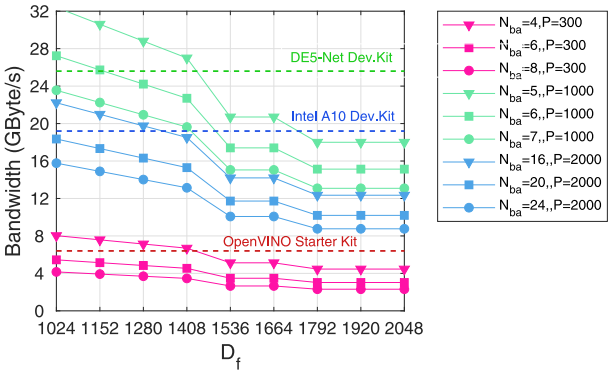

Fig. 12. Estimated external memory bandwidth when running AlexNet model. P denotes the target performance (GOP/s) under which the bandwidths are evaluated. The dotted lines represent the upper external memory bandwidth limits of the FPGA boards used in this article.

quickly due to an underutilization of the accumulator array for the last few layers that has small feature map sizes in dimension r'. Normally, we want to choose the largest reasonable N_r to maximize parallelism of the dataflow. Therefore, the flow selects 14 as the best configuration for parameter N_r. For VGG16, the performance gain raises back to a level very close to the upper limit at $N_r = 28$, so it is selected as a suboptimal value which will also be considered during design exploration. As has been discussed in Section IV-B, each of the feature map line-buffer should be large enough to hold one line of the prefetch window, which requires $D_f \geq S_{\text{win}} \times S \times N$ and $S_{\text{win}} \geq K'$. The minimum D_f can then be determined for the specific CNN model and the final value will be adjusted based on bandwidth analysis. After this step, the design parameters N_g, N_r, D_w, and D_q are fixed.

In the following step, several rounds of fast compilation based on OpenCL design codes are carried out for the target FPGA device, and hardware resource utilization information, including logic, DSP, and on-chip memory usage, are collected. Design constants which characterize the hardware costs of the accelerator are then solved based on the resource information and the *resource requirement model*. Fig. 13 gives an example of the calculated models for the AlexNet neural network on the Stratix-V GXA7 device. The measured R^2 of linear regression for the three hardware resources are all larger than 98%, which proves the correctness of the proposed mathematical models for hardware cost estimation. Note that the fast compilations are based on Intel's OpenCL Offline Compiler [32] and the resource data are estimated value rather than the actual one after FPGA placement. Base on our experiments, there is a 10% deviation between the estimated and the actual logic utilization. However, the DSP and memory resource data are accurate since the placement and routing procedures generally do not affect the utilization of these two types of hardware resources. One important advantage of adopting fast compilation for hardware cost evaluation is that it enables very quick generation of the resource models. For instance, a compilation of 20 design points for different pairs of N_{ba} and N_m only takes around three hours to finish and the whole process can be automatically controlled by a Python script.

In the final step, the highest attainable performance is explored in the N_m-N_{ba} design space by using the defined *performance and resource models* as shown in Fig. 14. Constraint of a full utilization of the DSP and memory resources is applied during exploration. However, a strict budget on logic resource (such as 80%) may lead to failures in placement and routing or large degradation in operating frequency. Therefore, several design candidates (usually one preferred and one alternative) with close logic utilization ratio are selected for final implementation. For instance, in Fig. 14, the two design points with configuration of $N_{ba} = 32, N_m = 2$ and $N_{ba} = 20, N_m = 3$ are saved by the flow and FPGA synthesis and mapping will be conducted for both design points. In this step, the external memory bandwidth is also calculated based on the *bandwidth model* to verify the input batch size N_{ba} and feature map buffer depth D_f have been correctly configured so that the required memory bandwidth under the performance target does not exceed the capacity of the FPGA board. As shown in Fig. 12, there is a minimum input batch size that should be met for a specific performance target. For example, if we are targeting a 2 TOP/s throughput goal for AlexNet model on the Intel A10 GX FPGA board on which a single DDR4 memory is attached to the FPGA device and is capable of transmitting data at the rate of 19.2 GB/s, the smallest input batch size that should be configured is 20. If the flow detects that the parameter N_{ba} does not meet

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

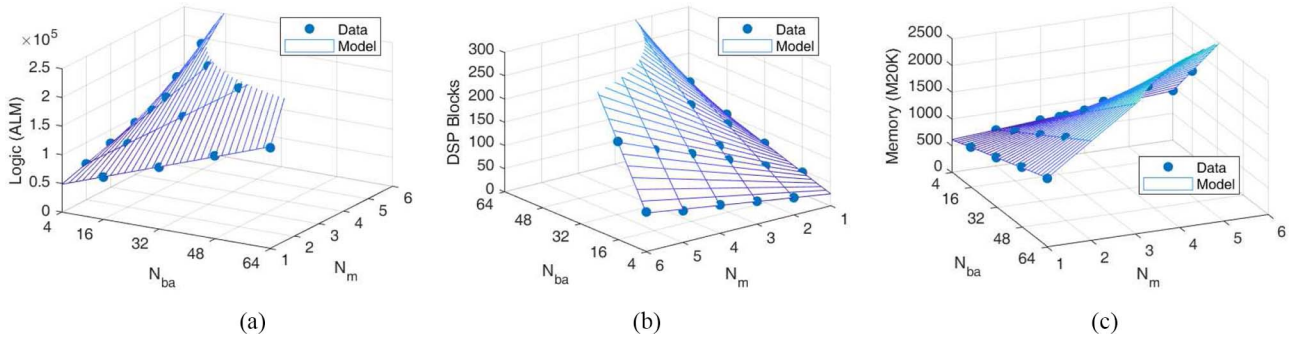

Fig. 13. Resource requirement models generated based on collected FPGA resource data from fast compilation of the OpenCL kernel codes. The target CNN model and FPGA device are AlexNet and Stratix-V GXA7, respectively. Other hardware parameters are configured as $N_r = 14, N_g = 8, D_f = 1024, D_w = 1024$, and $D_q = 36$. (a) Logic resource (ALM). (b) DSP blocks. (c) Memory usage (M20K).

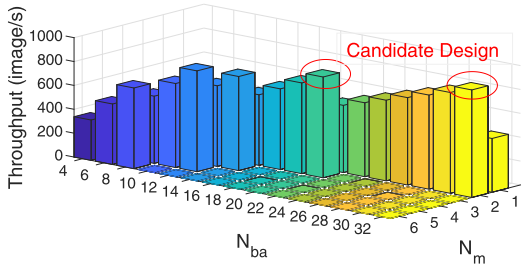

Fig. 14. Exploration for the highest attainable throughput. The example shown is for AlexNet with $N_{knl} = 14$, $N_g = 8$, and Freq = 200 MHz. The logic utilization budget is set as 75% and the target device is Stratix-V-A7 FPGA.

the bandwidth requirement, it will enlarge D_f to relieve the bandwidth pressure.

VI. Implementation Results

A. Experimental Setup

To evaluate the performance of the proposed accelerator, we have implemented the design on three different FPGA platforms, including OpenVINO Starter Kit (Cyclone-V), DE5-Net Development Kit (Stratix-V), and Intel A10 GX FPGA Development Kit (Arria-10), which covered a wide range of FPGA devices targeting from embedded application to high-performance computing. The hardware specifications of the FPGA boards are summarized in Table IV. The FPGA boards were installed on the PCIE 3.0×8 slots of a workstation, which was equipped with an Intel i7-9700K CPU and 64-GB memory. A high-level-synthesis (HLS)-based FPGA design methodology was adopted in hardware implementation. The proposed architecture was coded in OpenCL kernels and compiled by using Intel FPGA OpenCL SDK v18.1.

B. Performance and Resource Analysis

The implemented designs were tested by running inference computation of three CNN benchmarks, including AlexNet, VGG16, and ResNet-50, on the ImageNet dataset and the hardware parameters configured for different FPGA platforms are summarized in Table V. In the final design, FPGA executed all convolution, FC, ReLU, and Eltwise layers, while the remaining layers, such as pooling, LRN, and softmax, were executed by the host program on CPU. For ResNet, the batch normalization parameters were folded into the convolutional or FC layer's weight and bias during quantization as suggested by Jacob *et al.* [30]. By adopting pipelined processing, the execution time of CPU were hidden by FPGA.

Fig. 15 reports the measured inference throughputs and hardware resource utilizations of the final designs. Similar to previous studies [16], [19], inference throughput was calculated as the total #*OP* for spatial convolution of the original (dense) model divided by the average inference time per image. We can see that the average throughput increased by around 5× on average when migrating the design from Cyclone-V to Stratix-V device and the improvement is around 2× when changing from Stratix-V to Arria-10 FPGA. For VGG, there is a 10% drop on the highest achievable working

TABLE IV
Comparison of the FPGA Development Boards Used

Board Name	OpenVINO Starter Kit	DE5-Net Dev. Kit	Intel A10 GX Dev. Kit
FPGA	Cyclone-V GXC9	Stratix-V GXA7	Arria-10 GX1150
Logic Elements[1]	113K	234K	427K
DSP Blocks	342	256	1518
On-chip Memory	14Mb	57Mb	66Mb
External Memory	1GB DDR3	2x2GB DDR3	2GB DDR4
DDR Bandwidth	6.4 GB/s	25.6 GB/s	19.2 GB/s

[1] In terms of ALM.

TABLE V
Design Parameters Configured for Final Implementation

OpenVINO Starter Kit							
CNN Model	N_r	N_g	N_m	N_{ba}	D_f	D_w	D_q
AlexNet	14	8	3	8	1024	1024	36
VGG	28	8	3	4	1536	2048	36
ResNet	14	4	2	8	2048	1024	36
DE5-Net Dev. Kit							
CNN Model	N_r	N_g	N_m	N_{ba}	D_f	D_w	D_q
AlexNet	14	8	3	20	1024	1024	36
VGG	14	8	3	20	2048	2048	36
ResNet	14	4	3	20	2048	1024	36
Intel A10 GX Dev. Kit							
CNN Model	N_r	N_g	N_m	N_{ba}	D_f	D_w	D_q
AlexNet	14	8	4	32	1024	1024	36
VGG	28	8	2	32	2048	2048	36
ResNet	14	4	3	32	2048	1024	36

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

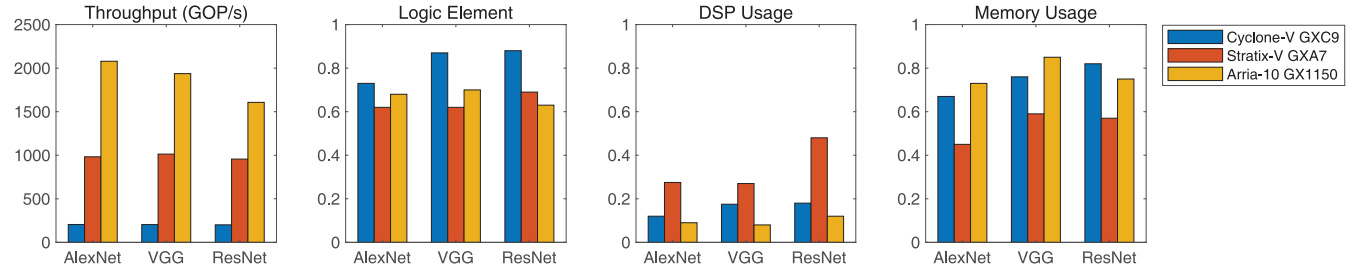

Fig. 15. Comparison of the implementation results for the final design on different FPGA devices.

TABLE VI
COMPARISON WITH STATE-OF-THE-ART FPGA CNN ACCELERATORS

	[15] SDConv	[14] SDConv	[16] FDConv	[19] FDConv	[23] SpConv	Proposed ABM-SpConv		
Benchmark Model	VGG16	VGG16	AlexNet	VGG16	AlexNet	ResNet-50	VGG16	AlexNet
FPGA Device	Zynq XC7Z020	Arria-10 GT1150	Arria-10 GX1150	Stratix-V GXA7	Zynq ZCU102	Cyclone-V GXC9	Stratix-V GXA7	Arria-10 GX1150
Freq. (MHz)	150	231	303	200	200	101	200	213
Model Precision (bit)	8 (fixed)	8-16 (fixed)	16 (float)	16 (fixed)	16 (fixed)	8 (fixed)	8 (fixed)	8 (fixed)
Batch Size	1	–	$< 15^2$	$< 15^2$	–	8	20	32
Logic Elements[1]	28K (52%)	313K (73%)	246K (58%)	107K (46%)	552K(92%)	100K (88%)	144K (62%)	291K (68%)
DSP Blocks	220 (100%)	1500 (98%)	1476 (97%)	256 (100%)	1144 (45%)	62 (18%[3])	69 (27%[3])	136 (9%)
On-Chip Memory[2]	75 (54%)	1668 (61%)	2487 (92%)	1377 (54%)	912 (48%)	1120 (97%)	2560 (100%)	2582 (95%)
Design Methodology	RTL	RTL	OpenCL	RTL	HLS	OpenCL	OpenCL	OpenCL
Throughput (GOP/s)	59.5	1171	1382	669.1	–	201	1013	2079
Arithmetic Perf. (GOP/s)	–	–	–	–	223.4	32	254	571

[1] Measured in ALM for Intel FPGA and LUT for Xilinx FPGA.
[2] Measured in M10K/M20K for Intel FPGA and BRAM for Xilinx FPGA.
[3] Measured in terms of available hard multipliers in the DSP blocks.

frequency when the accelerator is implemented on the Arria-10 device. The frequency drop was caused by increased logic and memory resource consumptions introduced by the larger feature map and weight buffers.

For ResNet, an extra layer of Eltwise is implemented in the MISC. unit, which have introduced extra amounts of 3% logic and 7% memory usages compared to the other two CNN models. The design with the preferred parameter setting of $N_{ba} = 32, N_m = 4$ could not be successfully mapped on the Arria-10 FPGA due to routing congestions, and the alternative design candidate ($N_{ba} = 32, N_m = 3$ as shown in Table V) with a 19% lower throughput was finally implemented. On average, the inference performance of the accelerator when running ResNet is 10% lower than the highest performance achieved when running the other two CNN models. The inefficiency mainly comes from the many $1 \times 1 \times N$ convolutional layers, whose small kernel sizes have led to low utilization of the pipeline.

As illustrated in Table I, previous CNN accelerators consumed all the on-chip DSP resources to conduct parallel MAC operations. In our design, only 15%–30% of the on-chip DSP blocks were used and the peak performance of accelerator was bounded by the logic resource on the Cyclone-V and Stratix-V platforms and by the on-chip memory on the Arria-10 device. Note that the Quartus Prime tool does not automatically pack two 16b-by-8b multipliers into a single DSP block until all the on-chip DSPs have been partially consumed when compiling for Cyclone-V and Stratix-V devices. Therefore, in Fig. 15, the DSP usage is measured in terms of all available hard multipliers to make the evaluation more accurate. The memory usage includes both M10K/M20K and MLAB blocks.

C. Comparison With State-of-the-Art

Table VI summarizes the comparison with state-of-the-art FPGA accelerators, which cover all three categories of convolution scheme, i.e., spatial convolution [14], [15], frequency domain convolution [16], [19], and SpConv [23]. We report the inference throughput as the primary metric for performance comparison. Lu *et al.* [23] only provided the effective number of MAC operations per second as the performance number, and in the comparison, they multiplied the inference throughput of other designs directly with the model sparsity ratio. We argue that this is not an appropriate approach. Because model sparsity only represents the reduction rate of the number of nonzero weights (parameters) rather than the computational workloads. For instance, we can see from Table III that the 92% sparsity of the ResNet-50 model (there is only one FC layer, so its effect on sparsity is marginal) only results in a 6.8× reduction on workload, whereas the reduction on model size is 9.5×. However, Lu *et al.* [23] did not provide any information on the workload of the pruned model and the reported data were all raw arithmetic performance numbers, which were generally calculated as $N_{\text{MAC}} \times 2 \times \text{Freq.} \times \text{Efficiency}$. Considering the proposed architecture is no longer based-on MAC units and the accumulator array is now the main performance bottleneck (i.e., its contribution to inference throughput is the same as MAC array),

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

we replace N_{MAC} with N_{acc} in the raw arithmetic performance formula to obtain a comparable number in terms of #OP/s in the last row of Table III as a secondary metric for performance comparison.

When compared to [19], our design achieves a 1.5× performance improvement and only uses around one fourth of the on-chip DSP resource when running the same CNN model on the same FPGA device. Even though the accelerator presented in [16] works at a much higher frequency, the proposed design still boosts the inference throughput by 50%. The most significant advantage of our approach is the 10× reduction in DSP utilization, which is the direct effect of the proposed new SpConv algorithm and the heterogeneous array architecture. On the other hand, due to the relaxed constraint on DSP resource, the proposed accelerator could utilize more computational resources (i.e., accumulators) to achieve further improvement in inference throughput. For instance, our design utilizes a number of 840 accumulators on the Stratix-V device, which is 1.6× of the total number of MAC units used by Zeng *et al.* [19]. Considering both schemes share a similar reduction in model workload, one can conclude that the proposed new architecture is the main contributor of the performance boost. The very latest study of [23] is also based on SpConv but the accelerator is implemented on a more advanced FPGA device (16-nm silicon technology). The proposed accelerator shares a similar arithmetic performance when implemented on the Stratix-V FPGA (28 nm) and achieves considerably 2.5× improvement over [23] when implemented on the Arria-10 device (20 nm). The savings on DSP blocks are 16.5× and 8.4×, respectively.

For low-cost embedded FPGA implementations, we compare our design to the work of [15]. If only targeting for CNN inference computation, the proposed accelerator boosts the throughput by 3.3×. When considering an integration of the inference accelerator with other functional units which also require large amount of DSP for computation, it is infeasible for the design of [15]. On the other hand, when we lower the logic resource usage of the proposed accelerator to the same level as [15], it can be estimated from the *performance model* that our design still has a 1.68× advantage on performance but only consumes 10% of the on-chip DSPs. It is clear that our approach is more flexible when building an SoC design with the CNN inference accelerator and other functional IPs on a single FPGA device. Moreover, the implementation result also suggests that it is even possible to cut down the number of on-chip DSP blocks and build new types of FPGA devices with even lower cost and energy consumption without sacrificing the capability of performing high performance CNN inference computation. Such low-cost FPGA devices are especially suitable for embedded application scenarios, such as IoT, robotics, and surveillance systems.

VII. Other Related Work

Study of [20] presented an energy-efficient accelerator that deployed sparse CNNs on a Artix-7 FPGA. Techniques presented in this article were resource utilization and power optimization orientated. The CNN model evaluated was of low complexity (0.44 GOP) and the performance achieved was only 31.79 GOP/s. Li *et al.* [22] reported an design framework which mapped sparse CNNs onto an FPGA accelerator. The reported performance was 271.6 GOP/s on a Zynq VC706 FPGA without further information disclosed on resource utilization and working frequency. The work of [21] presented an algorithm-hardware co-design scheme to improve the efficiency of sparse convolutional layers executed in hardware. A structurally pruned AlexNet model was accelerated on a Virtex VC707 board and only the execution efficiency (64.5%) was disclosed, which was also lower than ours (75% on average). DNNDK is a commercial deep neural network development toolchain released by Xilinx and it can deliver state-of-the-art performance when used to deployment CNN-based applications. Fang *et al.* [25] utilized DNNDK to implement a light-weight object detection algorithm (SSD) on a Xilinx ZU9 FPGA. The reported throughput were 2.4 and 12.4 TOP/s for dense and sparse CNN models, respectively. For the sparse model, the total #OP was pruned by 6.7× which was close to our scheme. However, the design utilized a number of 2027 DSP blocks, which was 15× larger than the proposed design on the Arria-10 device.

VIII. Conclusion

This article presented a high-throughput FPGA accelerator design which targeted efficient implementation of sparse CNN with low DSP costs. A new SpConv scheme along with an efficient hardware architecture were developed. By transforming the design space of FPGA-based accelerator from MAC-bound to accumulator-bound, our design successfully achieved 1.5× inference throughput improvement and 4× saving on DSP resource over the best frequency domain convolution based accelerator on the same FPGA device.

For future studies, we are planning to develop a latency-optimized hardware architecture for application scenarios in which the time to react on a single image/message is very important. The detailed works may include a redesign of the on-chip data buffering circuits and a new parallel dataflow targeting reduced latency.

Acknowledgment

The authors would like to thank the anonymous reviewers for their valuable comments and constructive suggestions, which helped in improving the readability of this article.

References

[1] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, San Diego, CA, USA, 2015, pp. 1–14.

[2] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol.105, no.12, pp. 2295–2329, Dec. 2017.

[3] T.-J. Yang, Y.-H. Chen, and V. Sze, "Designing energy-efficient convolutional neural networks using energy-aware pruning," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit. (CVPR)*, Honolulu, HI, USA, 2017, pp. 6071–6079.

[4] J. Fowers *et al.*, "A configurable cloud-scale DNN processor for real-time AI," *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Architect. (ISCA)*, Los Angeles, CA, USA, 2018, pp. 1–14.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.

[5] K. Y. Guo *et al.*, "Angel-eye: A complete design flow for mapping CNN onto embedded FPGA," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 1, pp. 35–47, Jan. 2018.

[6] X. Shi *et al.*, "HERO: Accelerating autonomous robotic tasks with FPGA," in *Proc. IROS*, Madrid, Spain, 2018, pp. 7766–7772.

[7] J. A. D. Cameron, P. Savoie, M. E. Kaye, and E. J. Scheme, "Design considerations for the processing system of a CNN-based automated surveillance system," *Expert Syst. Appl.*, vol.136, pp. 105–114, Dec. 2019.

[8] S. Du, T. Huang, J. Hou, S. Song, and Y. Song, "FPGA based acceleration of game theory algorithm in edge computing for autonomous driving," *J. Syst. Architect.*, vol. 93, pp. 33–39, Feb. 2019.

[9] N. Suda *et al.*, "Throughput-optimized OpenCL-based FPGA accelerator for large-scale convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, New York, NY, USA, 2016, pp. 16–25.

[10] Y. F. Ma, N. Suda, Y. Cao, J.-S. Seo, and S. Vrudhula, "Scalable and modularized RTL compilation of convolutional neural networks onto FPGA," in *Proc. 26th Int. Conf. Field Program. Logic Appl. (FPL)*, Lausanne, Switzerland, 2016, pp. 1–8.

[11] D. Wang, K. Xu, and D. Jiang, "PipeCNN: An OpenCL-based open-source FPGA accelerator for convolution neural networks," in *Proc. Int. Conf. Field Program. Technol. (ICFPT)*, Melbourne, VIC, Australia, 2017, pp. 279–282.

[12] Y. F. Ma, Y. Cao, S. Vrudhula, and J.-S. Seo, "An automatic RTL compiler for high-throughput FPGA implementation of diverse deep convolutional neural networks," in *Proc. 27th Int. Conf. Field Program. Logic Appl. (FPL)*, Ghent, Belgium, 2017, pp. 1–8.

[13] Y. F. Ma, Y. Cao, S. Vrudhula, and J.-S. Seo, "Optimizing loop operation and dataflow in FPGA acceleration of deep convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, New York, NY, USA, 2017, pp. 45–54.

[14] X. C. Wei *et al.*, "Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs," in *Proc. 54th ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, Austin, TX, USA, 2017, pp. 1–6.

[15] K. Y. Guo *et al.*, "From model to FPGA: Software-hardware co-design for efficient neural network acceleration," in *Proc. IEEE Hot Chips 28th Symp. (HCS)*, Cupertino, CA, USA, 2016, pp. 1–27.

[16] U. Aydonat, S. O'Connell, D. Capalija, A. C. Ling, and G. R. Chiu, "An OpenCL deep learning accelerator on Arria 10," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, New York, NY, USA, 2017, pp. 55–64.

[17] J. C. Yu *et al.*, "Instruction driven cross-layer CNN accelerator with winograd transformation on FPGA," in *Proc. Int. Conf. Field Program. Technol. (ICFPT)*, Melbourne, VIC, Australia, 2017, pp. 227–230.

[18] H. Zeng, C. Zhang, and V. Prasanna, "Fast generation of high throughput customized deep learning accelerators on FPGAs," in *Proc. Int. Conf. ReConFigurable Comput. FPGAs (ReConFig)*, Cancun, Mexico, 2017, pp. 1–8.

[19] H. Zeng, R. Chen, C. Zhang, and V. Prasanna, "A framework for generating high throughput CNN implementations on FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays (FPGA)*, New York, NY, USA, 2018, pp. 117–126.

[20] A. Page, A. Jafari, C. Shea, and T. Mohsenin, "SPARCNet: A hardware accelerator for efficient deployment of sparse convolutional networks," *J. Emerg. Technol. Comput. Syst*, vol. 13, no. 3, pp. 1–31, May 2017.

[21] B. Li, W. Wen, J. Mao, S. Li, Y. Chen, and H. H. Li, "Running sparse and low-precision neural network: When algorithm meets hardware," in *Proc. 23rd Asia South Pac. Design Autom. Conf. (ASP-DAC)*, Jeju, South Korea, 2018, pp. 534–539.

[22] S. Li, W. Wen, Y. Wang, S. Han, Y. Chen, and H. Li, "An FPGA design framework for CNN sparsification and acceleration," in *Proc. IEEE 25th Annu. Int. Symp. Field Program. Custom Comput. Mach. (FCCM)*, Napa, CA, USA, 2017, p. 28.

[23] L. Q. Lu, J. Xie, R. Huang, J. Zhang, W. Lin, and Y. Liang, "An efficient hardware accelerator for sparse convolutional neural networks on FPGAs," in *Proc. IEEE 25th Annu. Int. Symp. Field Program. Custom Comput. Mach. (FCCM)*, San Diego, CA, USA, 2019, pp. 17–25.

[24] D. Wang, K. Xu, Q. Jia, and S. Ghiasi, "ABM-SpConv: A novel approach to FPGA-based acceleration of convolutional neural network inference," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, Las Vegas, NV, USA, 2019, pp. 1–6.

[25] S. Fang *et al.*, "Real-time object detection and semantic segmentation hardware system with deep learning networks," in *Proc. Int. Conf. Field Program. Technol. (FPT)*, Naha, Japan, 2018, pp. 389–392.

[26] P. Gysel, J. Pimentel, M. Motamedi, and S. Ghiasi, "Ristretto: A framework for empirical study of resource-efficient inference in convolutional neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 11, pp. 5784–5789, Nov. 2018.

[27] Y. Gao, Z. Liu, and D. Wang, "Error models of finite word length arithmetic in CNN accelerator design," in *Proc. Visual Commun. Image Process. (VCIP)*, Chengdu, China, 2016, pp. 1–4.

[28] S. Han, H. Z. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman codings," in *Proc. ICLR*, San Juan, Puerto Rico, 2016, pp. 1–14.

[29] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," in *Proc. ICLR*, Toulon, France, 2017, pp. 1–17.

[30] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vision Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 2704–2713.

[31] S. Anwar, K. Hwang, and W. Sung, "Structured pruning of deep convolutional neural networks," *J. Emerg. Technol. Comput. Syst.*, vol. 13, no. 32, pp. 1–18, 2017.

[32] *Intel FPGA SDK for OpenCL Pro Edition Programming Guide V18.1*, Intel Company, Santa Clara, CA, USA, 2018. [Online]. Available: https://www.intel.com/

Dong Wang (Member, IEEE) received the M.S. and Ph.D. degrees in electronic engineering from Xi'an Jiaotong University, Xi'an, China, in 2006 and 2010, respectively.

He is an Associate Professor with the Institute of Information Science, Beijing Jiaotong University, Beijing, China. He was a Visiting Scholar with the Department of Electrical and Computer Engineering, University of California at Davis, Davis, CA, USA, from 2018 to 2019. His research interests include computer arithmetic for reconfigurable devices and high performance and energy efficient computing architectures for embedded and machine learning applications.

Ke Xu received the B.S. degree from the Hefei University of Technology, Hefei, China, in 2016. He is currently pursuing the Ph.D. degree with the Institute of Information Science, Beijing Jiaotong University, Beijing, China.

His research interests include neural network compression, high performance computing architectures for embedded applications, and computer vision.

Jingning Guo received the B.S. degree in information and computing science from Shanxi University, Taiyuan, China, in 2018. She is currently pursuing the M.E. degree in electronics and communication engineering with the School of Computer and Information Technology, Beijing Jiaotong University, Beijing, China.

Her current research interest includes pruning algorithm of deep convolutional neural network model.

Soheil Ghiasi (Senior Member, IEEE) received the B.S. degree from the Sharif University of Technology, Tehran, Iran, in 1998, and the M.S. and Ph.D. degrees in computer science from the University of California at Los Angeles, Los Angeles, CA, USA, in 2002 and 2004, respectively.

He is a Professor of Electrical and Computer Engineering with the University of California at Davis, Davis, CA, USA. His research interests include design methodologies and design automation techniques for embedded systems, with particular emphasis on systems that find applications in growing areas, such as machine intelligence and human health.

Prof. Ghiasi has served on the organizing and technical program committees of numerous conferences in the broad area of embedded computing systems. He is a Senior Member of ACM.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 07:07:40 UTC from IEEE Xplore. Restrictions apply.